MIMIC-IV Data Analysis via medicalcoder

Supplement to JAMIA Open Manuscript

Table of contents

List of Figures

List of Tables

1 Objective and Setup

This supplement provides reproducible validation and benchmarking of `medicalcoder::comorbidities()` using MIMIC-IV v3.1 data, including algorithmic agreement with established implementations and demonstration of longitudinal flagging behavior.

2 Materials and Methods

2.1 Data

We use the MIMIC-IV v3.1 Data (Johnson et al. 2024; Johnson et al. 2023; Goldberger et al. 2000) as the foundation for these examples.

The MIMIC-IV v3.1 dataset is a large, publicly available clinical database containing de-identified health data from patients admitted to intensive care units at Beth Israel Deaconess Medical Center. It includes detailed information on diagnoses, procedures, laboratory results, medications, and hospital encounters, with ICD-9-CM and ICD-10-CM/PCS codes available for billing and comorbidity research. MIMIC-IV is widely used for method development, validation, and reproducible research in clinical informatics and outcomes research.

The MIMIC-IV v3.1 data set can be obtained so long as you meet the requirements set by PhysioNet[1].

2.2 Comorbidity Algorithms

Within the *medicalcoder* package are implementations for several variants of the Charlson, Elixhauser, and PCCC comorbidity algorithms, (Table 1) (Quan et al. 2005, 2011; Glasheen et al. 2019; Healthcare Research and (AHRQ) 2025; Feudtner et al. 2014; Feinstein et al. 2024; DeWitt et al. 2025; Feinstein et al. 2018; *Complex Chronic Conditions Version 3.0* 2024).

For this supplement we focus on `charlson_quan2005`, `elixhauser_quan2005`, `pccc_v2.0`, and `pccc_3.1`.

NOTE: The MIMIC-IV data consists of only adult patients. Although PCCC was designed for pediatric populations, adult MIMIC-IV data are used here solely to demonstrate algorithmic implementation.

2.3 Alternative Utilities

We compared the results from `medicalcoder::comorbidities()` to the Charlson comorbidities flagged by the MIMIC-IV code, and the R package *comorbidity* (Gasparini 2018). For Elixhauser we will compare results between `medicalcoder::comorbidities()` and `comorbidity::comorbidity()`. Lastly, we will compare PCCC version 2 results between `medicalcoder::comorbidities()` and the R package *pccc* (DeWitt et al. 2025) call `pccc::ccc()`.

[1] https://physionet.org/content/mimiciv/3.1/

Table 1: Comorbidity methods implemented in *medicalcoder*.

Comorbidity Method	Notes
Charlson	
charlson_deyo1992	ICD-9 codes defined in Table 1 of Quan et al. (2005)
charlson_quan2005	ICD-9 and ICD-10 defined in Table 1 of Quan et al. (2005); index scoring as reported in Table 2 of Quan et al. (2011)
charlson_quan2011	ICD-9 and ICD-10 defined in Table 1 of Quan et al. (2005); index scoring as reported in Table 2 of Quan et al. (2011)
charlson_cdmf2019	ICD-9 and ICD-10 defined in [@glasheen2019]
Elixhauser	
elixhauser_elixhauser1988	ICD-9 codes defined in Table 2 of Quan et al. (2005)
elixhauser_ahrq_web	ICD-9 codes defined in Table 2 of Quan et al. (2005)
elixhauser_quan2005	ICD-9 and ICD-10 defined in Table 2 of Quan et al. (2005)
elixhauser_ahrq2022	ICD-10 codes from AHRQ for fiscal year 2022
elixhauser_ahrq2023	ICD-10 codes from AHRQ for fiscal year 2023
elixhauser_ahrq2024	ICD-10 codes from AHRQ for fiscal year 2024
elixhauser_ahrq2025	ICD-10 codes from AHRQ for fiscal year 2025
elixhauser_ahrq2026	ICD-10 codes from AHRQ for fiscal year 2026
elixhauser_ahrq_icd10	Any ICD-10 code from all elixhauser_ahrqYYYY
PCCC	
pccc_v2.0	ICD-9 and ICD-10 diagnostic and procedure codes consistent with pccc::ccc()
pccc_v2.1	Extended set of pccc_v2.0 codes based on pccc::ccc() and supplements to Feudtner et al. (2014)
pccc_v3.0	ICD-9 and ICD-10 diagnostic and procedure codes consistent with SAS code from Children's Hospital Assoication
pccc_v3.1	Extended set of pccc_v3.0 codes based on the supplements to Feinstein et al. (2024)

3 Results

3.1 MIMIC-IV Data Prep

We have stored the path to the MIMIC-IV data as a system variable on our machines.

```
# verify that you are looking at version 3.1 of the MIMIC-IV data
stopifnot(
  identical(
    scan(
      file = file.path(Sys.getenv("MIMICIVDATA"), "CHANGELOG.txt"),
      what = "character", sep = "\n", quiet = TRUE)[1],
    "This is the change log for MIMIC-IV v3.1."
  )
)
```

We only need four of the MIMIC-IV hospital datasets: `admissions`, `patients`, `diagnoses_icd`, and `procedures_icd`.

We will do some light formatting before using `medicalcoder::comorbidities()` to flag comorbidities. Start by importing the `admissions`, `patients`, `diagnoses_icd`, and `procedures_icd`, datasets.

Table 2 gives the sha256 checksums for these files Each file checksum has been verified.

Table 2: SHA256 Check sums for files in the MIMIC-IV v3.1 data.

sha256sum	file
a9584ed88e9ed664a2f66f86a5cf9fd175c8bb0af50e3b6115598b19e978384e	hosp/admissions.csv.gz
47665a41b2a3ad990d6f314062d5bd6d29d467b0fd402dd94662044ec8b073d2	hosp/diagnoses_icd.csv.gz
4c7507de7ecad8c5ba7647ea4d26018e88ff6672c5fc22ab59c2a5697d687cdd	hosp/patients.csv.gz
3271397d0517131f84399b698cbb0c2f435aaab0f873a3806e13b097661596f7	hosp/procedures_icd.csv.gz

```
mimicivdata <-
  list.files(
    path = file.path(Sys.getenv("MIMICIVDATA"), "hosp"),
    pattern = "(admissions|patients|.+_icd)\\.csv\\.gz",
    full.names = TRUE
  )
names(mimicivdata) <- sub("\\.csv\\.gz$", "", basename(mimicivdata))

mimicivdata <- lapply(mimicivdata, data.table::fread)
```

We will build an additional data set for age (in years).

```
mimicivdata$ages <-
  merge(
    x = mimicivdata$admissions[, .(subject_id, hadm_id, admittime)],
    y = mimicivdata$patients[,   .(subject_id, anchor_age, anchor_year)],
    all = FALSE,
    by = "subject_id"
  )
mimicivdata$ages[
  ,
  anchor_year := as.POSIXct(paste0(anchor_year, "-01-01"), format = "%Y-%m-%d")
]
mimicivdata$ages[
  ,
  age :=
    anchor_age +
    as.numeric(difftime(admittime, anchor_year, units = "days")) / 365.25
]
```

For the calls to `medicalcoder::comorbidities()` we can have all the ICD codes in one data.frame (data.table will be used here).

```
mimicivdata$icd <-
  data.table::rbindlist(
    list(dx = mimicivdata$diagnoses_icd, pr = mimicivdata$procedures_icd),
    idcol = "dx",
    use.names = TRUE,
    fill = TRUE
  )

# set the diagnoses indicator to an integer as expected for input to
# medicalcoder::comorbidities()
mimicivdata$icd[, dx := as.integer(dx == "dx")]
```

We put all the parts together into one data.frame (data.table)

```
mimicivDT <-
  merge(
    x = mimicivdata$patients[, .(subject_id)],
    y = mimicivdata$ages[, .(subject_id, hadm_id, age)],
    all = TRUE,
    by = c("subject_id")
  )

mimicivDT <-
  merge(
    x = mimicivdata$icd,
    y = mimicivDT,
    all = TRUE,
    by = c("subject_id", "hadm_id")
  )

mimicivDT <-
  merge(
    x = mimicivDT,
    y = mimicivdata$admissions[, .(subject_id, hadm_id, admittime)],
    all = TRUE,
    by = c("subject_id", "hadm_id")
  )

# set keys and order the data by subject id, admittime, hadm_id
data.table::setkey(mimicivDT, subject_id, admittime, hadm_id)

# set an encounter sequence variable. This works because the data has been
```

```
# sorted by subject_id, admittime.  NOTE: admittime can be used in the calls to
# medicalcoder::comorbidities() but it there is additional overhead and
# computational expense when sorting and joining by POSIXct variable.  The
# enc_seq achieves the same utility with lower computational overhead.
mimicivDT[, enc_seq := cumsum(!duplicated(hadm_id)), by = .(subject_id)]

# For some of the methods, only one id variable is allowed so we create a
# composite id:
mimicivDT[, cid := paste(subject_id, enc_seq, sep = "__")]
```

```
str(mimicivDT)
## Classes 'data.table' and 'data.frame':   7365770 obs. of  11 variables:
##  $ subject_id : int  10000032 10000032 10000032 10000032 10000032 10000032 10000032 10000
##  $ hadm_id    : int  22595853 22595853 22595853 22595853 22595853 22595853 22595853 22595
##  $ dx         : int  1 1 1 1 1 1 1 1 0 1 ...
##  $ seq_num    : int  1 2 3 4 5 6 7 8 1 1 ...
##  $ icd_code   : chr  "5723" "78959" "5715" "07070" ...
##  $ icd_version: int  9 9 9 9 9 9 9 9 9 9 ...
##  $ chartdate  : IDate, format: NA NA ...
##  $ age        : num  52.3 52.3 52.3 52.3 52.3 ...
##  $ admittime  : POSIXct, format: "2180-05-06 22:23:00" "2180-05-06 22:23:00" ...
##  $ enc_seq    : int  1 1 1 1 1 1 1 1 1 2 ...
##  $ cid        : chr  "10000032__1" "10000032__1" "10000032__1" "10000032__1" ...
##  - attr(*, ".internal.selfref")=<externalptr>
##  - attr(*, "sorted")= chr [1:3] "subject_id" "admittime" "hadm_id"
```

There are 7,365,770 total rows of data for 364,627 unique subjects and a total of 687,203 unique hospital encounters.

3.2 Flagging comorbidities

In the following we will outline how to apply the comorbidities methods to the `mimicivDT` data set. We will define some wrapper functions to simplify calls. We will return the results and time how long it takes to apply the methods. We will flag the following comorbidities:

- Charlson (Quan et al. 2005): `method = 'charlson_quan2005'`
- Elixhauser (Quan et al. 2005): `method = 'elixhauser_quan2005'`
- PCCC v2 (Feudtner et al. 2014): `method = 'pccc_v2.0'`
- PCCC v3 (Feinstein et al. 2024) with additional codes: `method = 'pccc_v3.1'`

For each of these comorbidity methods we will flag on a per-encounter basis and a longitudinal basis within a patient record.

3.2.1 `medicalcoder::comorbidities()`

The call to `medicalcoder::comorbidities()` has been designed to make calling multiple comorbidity methods and multiple flagging methods easy. For this use case it would be programmatically preferable to use a simple wrapper about `medicalcoder::comorbidities()` to simplify the forthcoming calls. However, we opt to repeat ourselves in the following calls for clarity and ease of selective reproduction by the reader.

3.2.1.1 Charlson

The following code chunk will apply the Charlson (Quan et al. 2005) comorbidities algorithms to each encounter in the data set.

```
medicalcoder_charlson_current <-
  medicalcoder::comorbidities(
    data        = mimicivDT,  # object inheriting data.frame
    icd.codes   = "icd_code", # character string name of icd codes in data
    id.vars     = c("subject_id", "enc_seq"),
    icdv.var    = "icd_version",
    dx.var      = "dx",
    poa         = 1L,    # consider all codes present on admission
    age.var     = "age", # in years, only relevant to charlson
    primarydx   = 0L,    # consider all diagnosis codes secondary diagnoses.
    method      = "charlson_quan2005",
    flag.method = "current" # default
  )
```

The next call will apply the Charlson (Quan et al. 2005) comorbidities to the MIMIC-IV v3.1 data within a subject's record. That is, a comorbidities flagged on on encounter i will also be flagged on encounters $i + 1$, $i + 2$, ...

Two important things to note about the following call.

1. Longitudinal results within a subject is returned when `flag.method = 'cumulative'`.
2. There is an expectation that there are at least two identification variables. The last one needs to be sortable variable. That is `id.vars = c("subject_id", "eqn_seq")` means that the data will be sorted by `subject_id` and `enc_seq` with `enc_seq = 1` considered to be occur before `enc_seq = 2`, etc. In the MIMIC-IV v3.1 data, the `hadm_id` are not sequential within a patient record, hence the construction of the `enc_seq` variable.

```
medicalcoder_charlson_cumulative <-
  medicalcoder::comorbidities(
    data        = mimicivDT,
    icd.codes   = "icd_code",
    id.vars     = c("subject_id", "enc_seq"),
    icdv.var    = "icd_version",
    dx.var      = "dx",
    poa         = 1L,
    age.var     = "age",
    primarydx   = 0L,
    method      = "charlson_quan2005",
    flag.method = "cumulative"
  )
```

3.2.1.2 Elixhauser

Applying Elixhauser requires a only a change to the `method` argument.

```
medicalcoder_elixhauser_current <-
  medicalcoder::comorbidities(
    data        = mimicivDT,
    icd.codes   = "icd_code",
    id.vars     = c("subject_id", "enc_seq"),
    icdv.var    = "icd_version",
    dx.var      = "dx",
    poa         = 1L,
    primarydx   = 0L,
    method      = "elixhauser_quan2005",
    flag.method = "current"
  )
```

```
medicalcoder_elixhauser_cumulative <-
  medicalcoder::comorbidities(
    data        = mimicivDT,
    icd.codes   = "icd_code",
    id.vars     = c("subject_id", "enc_seq"),
    icdv.var    = "icd_version",
    dx.var      = "dx",
    poa         = 1L,
    primarydx   = 0L,
    method      = "elixhauser_quan2005",
```

```
    flag.method = "cumulative"
  )
```

3.2.1.3 PCCC v2.0

PCCC v2.0 has been built to be consistent with the output from `pccc::ccc()` and is called used `method = "pccc_v2.0"`. Another option is `pccc_v2.1` which has additional ICD codes based on our interpretation of the intention of the mappings based on the supplements to Feudtner et al. (2014).

```
medicalcoder_pcccv2.0_current <-
  medicalcoder::comorbidities(
    data        = mimicivDT,
    icd.codes   = "icd_code",
    id.vars     = c("subject_id", "enc_seq"),
    icdv.var    = "icd_version",
    dx.var      = "dx",
    poa         = 1L,
    method      = "pccc_v2.0",
    flag.method = "current"
  )
```

```
medicalcoder_pcccv2.0_cumulative <-
  medicalcoder::comorbidities(
    data        = mimicivDT,
    icd.codes   = "icd_code",
    id.vars     = c("subject_id", "enc_seq"),
    icdv.var    = "icd_version",
    dx.var      = "dx",
    poa         = 1L,
    age.var     = "age",
    method      = "pccc_v2.0",
    flag.method = "cumulative"
  )
```

3.2.1.4 PCCC v3

PCCC v3 (Feinstein et al. 2024) differs from PCCC v2 (Feudtner et al. 2014) in two major ways: (1) Updated mappings between ICD codes and conditions. (2) How conditions flagged by technology dependence are handled. If the only reason for a conditions to be flagged is due to technology dependence then under PCCC v3 the condition *will not* be flagged. At least one

non-technology dependent code needs to be in the patient record for the technology-dependent conditions to also be flagged. This makes longitudinal flagging particularly difficult. Examples to follow.

```
medicalcoder_pcccv3.1_current <-
  medicalcoder::comorbidities(
    data        = mimicivDT,
    icd.codes   = "icd_code",
    id.vars     = c("subject_id", "enc_seq"),
    icdv.var    = "icd_version",
    dx.var      = "dx",
    poa         = 1L,
    method      = "pccc_v3.1",
    flag.method = "current"
  )
```

```
medicalcoder_pcccv3.1_cumulative <-
  medicalcoder::comorbidities(
    data        = mimicivDT,
    icd.codes   = "icd_code",
    id.vars     = c("subject_id", "enc_seq"),
    icdv.var    = "icd_version",
    dx.var      = "dx",
    poa         = 1L,
    age.var     = "age",
    method      = "pccc_v3.1",
    flag.method = "cumulative"
  )
```

3.2.2 MIMIC-IV Code

We use some of the SQL code for applying the Charlson comorbidity algorithm to the MIMIC-IV data provided by the MIT Laboratory for Computational Physiology (MIT_LCP) on GitHub. We stored the path to a local copy of the repository in the system environmental variable `MIMICIVCODE`.

The SQL code for applying the Charlson comorbidities and checksum:

```
mimiciv_charlson_sql <-
  file.path(Sys.getenv("MIMICIVCODE"), "concepts", "comorbidity", "charlson.sql")

# checksum
```

```
stopifnot(
  substr(
    system(
      sprintf("sha256sum %s", mimiciv_charlson_sql),
      intern = TRUE
    ),
  start = 1L,
  stop = 64L
  ) == "5b797b673ace4dcbc6f686848d24371382f49ac61fcc39101f09c3b42969d801"
)
```

The SQL code is expected to run on Google Big Query. We make a few small modifications to the code so we can evaluate the SQL locally via RSQLite.

```
mimiciv_charlson_sql <-
  scan(
    file = mimiciv_charlson_sql,
    what = "character",
    sep = "\n",
    quiet = TRUE
  )

# modify the query to work in SQLite
mimiciv_charlson_sql <-
  gsub(
    pattern = "physionet-data.mimiciv_hosp.admissions",
    replacement = "admissions",
    x = mimiciv_charlson_sql,
    fixed = TRUE
  )

mimiciv_charlson_sql <-
  gsub(
    pattern = "physionet-data.mimiciv_hosp.diagnoses_icd",
    replacement = "diagnoses",
    x = mimiciv_charlson_sql,
    fixed = TRUE
  )

mimiciv_charlson_sql <-
  gsub(
    pattern = "physionet-data.mimiciv_derived.age",
```

```
    replacement = "ages",
    x = mimiciv_charlson_sql,
    fixed = TRUE
  )

mimiciv_charlson_sql <-
  gsub(
    pattern = "GREATEST",
    replacement = "MAX",
    x = mimiciv_charlson_sql,
    fixed = TRUE
  )

mimiciv_charlson_sql <- paste(mimiciv_charlson_sql, collapse = "\n")
```

We set up an SQL database in memory and evaluate the code.

```
con <- odbc::dbConnect(drv = RSQLite::SQLite(), dbname = ":memory:")

# add data to the data base
odbc::dbWriteTable(conn = con, name = "diagnoses",  value = mimicivdata$diagnoses)
odbc::dbWriteTable(conn = con, name = "admissions", value = mimicivdata$admissions)
odbc::dbWriteTable(conn = con, name = "patients",   value = mimicivdata$patients)
odbc::dbWriteTable(conn = con, name = "ages",       value = mimicivdata$ages)

# Run the query and record the time required to do so
mimiciv_charlson_current <- odbc::dbGetQuery(con, mimiciv_charlson_sql)
data.table::setDT(mimiciv_charlson_current)

# Note: not all subject, enc_seq (hadm_id) values are in the return.  Add the
# needed rows # and set all the flags to zero
stopifnot(
  nrow(unique(mimicivDT[, .(subject_id, hadm_id, enc_seq)])) >
  nrow(mimiciv_charlson_current)
)

mimiciv_charlson_current <-
  merge(
    x = unique(mimicivDT[, .(subject_id, hadm_id, enc_seq)]),
    y = mimiciv_charlson_current,
    all = TRUE,
    by = c("subject_id", "hadm_id")
```

```
  )

for (j in names(mimiciv_charlson_current)) {
  if (!(j %in% c("subject_id", "hadm_id", "enc_seq"))) {
    idx <- which(is.na(mimiciv_charlson_current[[j]]))
    mimiciv_charlson_current[[j]][idx] <- 0L
  }
}

# close DB connection
odbc::dbDisconnect(conn = con)
```

3.2.3 `comorbidity::comorbidity()`

The R package *comorbidity* (Gasparini 2018) is a widely used and referenced tool for applying the Charlson and Elixhauser comorbidities. One consideration for using `comorbidity::comorbidity()` is that ICD-9 or ICD-10 codes are used independently. There are 30921 subjects with both ICD-9 and ICD-10 codes in their records, and 11 encounters with both ICD versions.

To flag the conditions appropriately we must split the data by ICD version, apply `comorbidity::comorbidity()` to each split, and then aggregate the results. Also, `comorbidity::comorbidity()` only takes one column for the id and thus we need to use `cid` for the id instead of the `c("subject_id", "enc_seq")` used in the calls to `medicalcoder::comorbidities()`. In the following work we will copy the data built for the encounter level flagging and extend for cumulative flagging as well.

The following is done using *comorbidity* version 1.1.0.

3.2.3.1 Charlson

```
comorbidity_charlson_icd9 <-
  comorbidity::comorbidity(
    x       = subset(mimicivDT, icd_version == 9 & dx == 1L),
    id      = "cid",
    code    = "icd_code",
    map     = "charlson_icd9_quan",
    assign0 = TRUE # set less severe comorbidities flags to 0 when more severe
                   # comorbidities is also flagged
  )

comorbidity_charlson_icd10 <-
```

```
  comorbidity::comorbidity(
    x       = subset(mimicivDT, icd_version == 10 & dx == 1L),
    id      = "cid",
    code    = "icd_code",
    map     = "charlson_icd10_quan",
    assign0 = TRUE
  )

comorbidity_charlson_current <-
  rbind(
    comorbidity_charlson_icd9,
    comorbidity_charlson_icd10
  )

# aggregate between the icd versions
comorbidity_charlson_current <-
  aggregate(. ~ cid, data = comorbidity_charlson_current, FUN = max)

# Note, Not all the cid are in comorbidity_charlson_current.  Especially for
# building the cumulative flags we need all the subjects and encounters
comorbidity_charlson_current <-
  merge(
    x = comorbidity_charlson_current,
    y = unique(mimicivDT[, .(cid)]),
    all = TRUE,
    by = "cid"
  )

# set NA values to zero
for (j in names(comorbidity_charlson_current)[-1]) {
  idx <- which(is.na(comorbidity_charlson_current[[j]]))
  comorbidity_charlson_current[[j]][idx] <- 0L
}

# apply the scoring, to use comorbidity::score the object needs to have certain
# attributes which were lost when aggregating over the icd versions.  Replace
# the attributes
attributes(comorbidity_charlson_current)[c("class", "variable.labels", "map")] <-
  attributes(comorbidity_charlson_icd9)[c("class", "variable.labels", "map")]

comorbidity_charlson_current[["score"]] <-
  comorbidity::score(
```

```
    x = comorbidity_charlson_current,
    weights = "quan",
    assign0 = TRUE
  )

# the score return is a numeric value, setting to integer to match the storage
# mode of medicalcoder
comorbidity_charlson_current[["score"]] <-
  as.integer(comorbidity_charlson_current[["score"]])

# expand the id columns and sort
data.table::setDT(comorbidity_charlson_current)
comorbidity_charlson_current[
  ,
  c("subject_id", "enc_seq") := data.table::tstrsplit(cid, "__")
  ][
  ,
  `:=`(subject_id = as.integer(subject_id), enc_seq = as.integer(enc_seq))
  ]
data.table::setkey(comorbidity_charlson_current, subject_id, enc_seq)
```

```
# For the cumulative flagging
comorbidity_charlson_cumulative <- data.table::copy(comorbidity_charlson_current)

# next we use cumsum to flag conditions moving forward in time
cumflag <- function(x) as.integer(cumsum(x) > 0)

for (j in names(comorbidity_charlson_cumulative)) {
  if (!(j %in% c("subject_id", "enc_seq", "cid"))) {
    e <- substitute(
          comorbidity_charlson_cumulative[, J := cumflag(J), by = .(subject_id)],
          list(J = as.name(j))
          )
    eval(e)
  }
}

# clean up flags by severity
comorbidity_charlson_cumulative[diab == 1 & diabwc == 1, diab := 0L]
comorbidity_charlson_cumulative[mld == 1 & msld == 1, mld := 0L]
comorbidity_charlson_cumulative[canc == 1 & metacanc == 1, canc := 0L]
```

```
# apply the scoring, to use comorbidity::score the object needs to have certain
# attributes which were lost when aggregating over the icd versions.  Replace
# the attributes
data.table::setDF(comorbidity_charlson_cumulative)
attributes(comorbidity_charlson_cumulative)[c("class", "variable.labels", "map")] <-
  attributes(comorbidity_charlson_icd9)[c("class", "variable.labels", "map")]

comorbidity_charlson_cumulative[["score"]] <-
  comorbidity::score(
    x = comorbidity_charlson_cumulative,
    weights = "quan",
    assign0 = TRUE
  )

# the score return is a numeric value, setting to integer to match the storage
# mode of medicalcoder
comorbidity_charlson_cumulative[["score"]] <-
  as.integer(comorbidity_charlson_cumulative[["score"]])

data.table::setDT(comorbidity_charlson_cumulative)
```

3.2.3.2 Elixhauser

Compared to the work done for flagging Charlson comorbidities, Elixhauser flagging requires changing the `map` argument in the calls to `comorbidity::comorbidity()` and there is no scoring to be done.

```
comorbidity_elixhauser_icd9 <-
  comorbidity::comorbidity(
    x       = subset(mimicivDT, icd_version == 9 & dx == 1L),
    id      = "cid",
    code    = "icd_code",
    map     = "elixhauser_icd9_quan",
    assign0 = TRUE
  )

comorbidity_elixhauser_icd10 <-
  comorbidity::comorbidity(
    x       = subset(mimicivDT, icd_version == 10 & dx == 1L),
    id      = "cid",
    code    = "icd_code",
    map     = "elixhauser_icd10_quan",
```

```
    assign0 = TRUE
  )

comorbidity_elixhauser_current <-
  rbind(
    comorbidity_elixhauser_icd9,
    comorbidity_elixhauser_icd10
  )

# aggregate between the icd versions
comorbidity_elixhauser_current <-
  aggregate(. ~ cid, data = comorbidity_elixhauser_current, FUN = max)

# Note, Not all the cid are in comorbidity_elixhauser_current.  Especially for
# building the cumulative flags we need all the subjects and encounters
comorbidity_elixhauser_current <-
  merge(
    x = comorbidity_elixhauser_current,
    y = unique(mimicivDT[, .(cid)]),
    all = TRUE,
    by = "cid"
  )

# set NA values to zero
for (j in names(comorbidity_elixhauser_current)[-1]) {
  idx <- which(is.na(comorbidity_elixhauser_current[[j]]))
  comorbidity_elixhauser_current[[j]][idx] <- 0L
}

# expand the id columns and sort
data.table::setDT(comorbidity_elixhauser_current)
comorbidity_elixhauser_current[
  ,
  c("subject_id", "enc_seq") := data.table::tstrsplit(cid, "__")
  ][
  ,
  `:=`(subject_id = as.integer(subject_id), enc_seq = as.integer(enc_seq))
  ]
data.table::setkey(comorbidity_elixhauser_current, subject_id, enc_seq)
```

```
# For the cumulative flagging
comorbidity_elixhauser_cumulative <- data.table::copy(comorbidity_elixhauser_current)
```

```
# next we use cumsum to flag conditions moving forward in time
cumflag <- function(x) as.integer(cumsum(x) > 0)

for (j in names(comorbidity_elixhauser_cumulative)) {
  if (!(j %in% c("subject_id", "enc_seq", "cid"))) {
    e <- substitute(
          comorbidity_elixhauser_cumulative[, J := cumflag(J), by = .(subject_id)],
          list(J = as.name(j))
          )
    eval(e)
  }
}

# clean up flags by severity?
comorbidity_elixhauser_cumulative[diabunc  == 1L & diabc    == 1L, diabunc  := 0L]
comorbidity_elixhauser_cumulative[hypunc   == 1L & hypc     == 1L, hypunc   := 0L]
comorbidity_elixhauser_cumulative[solidtum == 1L & metacanc == 1L, solidtum := 0L]
```

3.2.4 `pccc::ccc()`

The R package *pccc* (DeWitt et al. 2025) implements PCCC v2 (Feudtner et al. 2014). As with `comorbidity::comorbidity()` and many other implementations of comorbidities, only one version of ICD is assessed at a time.

The expected input structure of the data for `pccc::ccc()` is a wide format, one row per encounter, one column for each ICD code.

The following uses *pccc* version 1.0.7.

```
mimicivDT_for_pccc <-
  split(mimicivDT, by = "icd_version") |>
  lapply(
    data.table::dcast,
    formula = cid ~ ifelse(dx == 1, "DX", "PR") + seq_num,
    value.var = "icd_code"
    )

pccc_pcccv2.0_current <-
  rbind(
    pccc::ccc(
      data = mimicivDT_for_pccc[["9"]],
      id = cid,
```

```
      dx_cols = grep("^DX", names(mimicivDT_for_pccc[["9"]]), value = TRUE),
      pc_cols = grep("^PR", names(mimicivDT_for_pccc[["9"]]), value = TRUE),
      icdv = 9
    )
    ,
    pccc::ccc(
      data = mimicivDT_for_pccc[["10"]],
      id = cid,
      dx_cols = grep("^DX", names(mimicivDT_for_pccc[["10"]]), value = TRUE),
      pc_cols = grep("^PR", names(mimicivDT_for_pccc[["10"]]), value = TRUE),
      icdv = 10
    )
  )

# aggregate over ICD version
pccc_pcccv2.0_current <-
  aggregate(. ~ cid, data = pccc_pcccv2.0_current, FUN = max)

# Not all the cid are in this data set.  add them and set the missing values to
# zeros.
pccc_pcccv2.0_current <-
  merge(
    x = pccc_pcccv2.0_current,
    y = unique(mimicivDT[, .(cid)]),
    all = TRUE,
    by = "cid"
  )

for (j in names(pccc_pcccv2.0_current)[-1]) {
  idx <- which(is.na(pccc_pcccv2.0_current[[j]]))
  pccc_pcccv2.0_current[[j]][idx] <- 0L
}

data.table::setDT(pccc_pcccv2.0_current)

pccc_pcccv2.0_current[,
  c("subject_id", "enc_seq") := data.table::tstrsplit(cid, "__")
  ][
  ,
  `:=`(
    subject_id = as.integer(subject_id),
    enc_seq = as.integer(enc_seq)
```

```
  )]

data.table::setkey(pccc_pcccv2.0_current, subject_id, enc_seq)
```

```
# cumulative flag
pccc_pcccv2.0_cumulative <- data.table::copy(pccc_pcccv2.0_current)

for (j in names(pccc_pcccv2.0_cumulative)) {
  if (!(j %in% c("cid", "subject_id", "enc_seq"))) {
    e <- substitute(pccc_pcccv2.0_cumulative[, J := cumflag(J), by = .(subject_id)],
      list(J = as.name(j)))
    eval(e)
  }
}
```

3.3 Consistency in Flags between Utilities

3.3.1 `medicalcoder::comorbidities()` vs MIMIC-IV Code

Comparing the results between `medicalcoder::comorbidities()` and MIMIC-IV code for Charlson comorbidities at the encounter level should note that the number of rows returned differed. In the code above that was rectified. `medicalcoder::comorbidities()` returned a row for each subject_id, enc_seq (hadm_id) pair, whereas the MIMIC-IV code did not return a row for each. This was resolved above and zeros imputed for the subject_id, hadm_id which were omitted from the `mimiciv_charlson_current` object.

We build one data.frame to make the comparisons between the two methods easier.

```
mdcr_v_mimiciv <-
  merge(
    x = medicalcoder_charlson_current,
    y = mimiciv_charlson_current,
    all = TRUE,
    by = c("subject_id", "enc_seq"),
    suffixes = c("_mdcr", "_mimiciv")
  )
```

There are three comorbidities where there are different levels of severity. `medicalcoder::comorbidities()` will set the less severe condition indicator to 0 when the more severe condition is flagged. The MIMIC-IV code returns a flag for both levels. Both `medicalcoder::comorbidities()` and MIMIC-IV Code only consider the more severe case in the index scoring. For the comorbidities

diabetes, cancer, and liver disease we will set the less sever flag to 0 if the more sever flag is present for the results from the MIMIC-IV Code.

```
mdcr_v_mimiciv[
  ,
  .(
    mdcr_dm    = sum(dm == 1L & dmc == 1L),
    mimiciv_dm = sum(diabetes_without_cc == 1L & diabetes_with_cc == 1L),
    mdcr_canc    = sum(mal == 1L & mst == 1L),
    mimiciv_canc = sum(malignant_cancer == 1L & metastatic_solid_tumor == 1L),
    mdcr_ld    = sum(mld == 1L & msld == 1L),
    mimiciv_ld = sum(mild_liver_disease == 1L & severe_liver_disease == 1L)
   )
]
##    mdcr_dm mimiciv_dm mdcr_canc mimiciv_canc mdcr_ld mimiciv_ld
##      <int>      <int>     <int>        <int>   <int>      <int>
## 1:       0      14068         0        19673       0      12551

mdcr_v_mimiciv[
  diabetes_without_cc == 1L & diabetes_with_cc == 1L,
  diabetes_without_cc := 0L
]

mdcr_v_mimiciv[
  malignant_cancer == 1L & metastatic_solid_tumor == 1L,
  malignant_cancer := 0L
]

mdcr_v_mimiciv[
  mild_liver_disease == 1L & severe_liver_disease == 1L,
  mild_liver_disease := 0L
]
```

We define the mapping between the columns returned from `medicalcoder::comorbidities()` and the MIMIC-IV Code:

```
mvm_columns <-
  data.table::fread(text = "
    medicalcoder   | mimicivcode
    aidshiv        | aids
    cebvd          | cerebrovascular_disease
    chf            | congestive_heart_failure
```

```
    copd            | chronic_pulmonary_disease
    dem             | dementia
    dm              | diabetes_without_cc
    dmc             | diabetes_with_cc
    hp              | paraplegia
    mal             | malignant_cancer
    mi              | myocardial_infarct
    mld             | mild_liver_disease
    msld            | severe_liver_disease
    mst             | metastatic_solid_tumor
    pud             | peptic_ulcer_disease
    pvd             | peripheral_vascular_disease
    rhd             | rheumatic_disease
    rnd             | renal_disease
    age_score_mdcr  | age_score_mimiciv
    cci             | charlson_comorbidity_index
  ")
```

The following `for` loop will check that the columns between `medicalcoder::comorbidities()` and MIMIC-IV Code are identical. If so, the columns are removed from the `mdcr_v_mimiciv` object, otherwise a message is printed.

```
for (i in seq_len(nrow(mvm_columns))) {
  j1 <- mvm_columns[["medicalcoder"]][i]
  j2 <- mvm_columns[["mimicivcode"]][i]
  e <- identical(mdcr_v_mimiciv[[j1]], mdcr_v_mimiciv[[j2]])
  if (e) {
    mdcr_v_mimiciv[[j1]] <- NULL
    mdcr_v_mimiciv[[j2]] <- NULL
  } else {
    print(sprintf("%s and %s are not identical", j1, j2))
  }
}
## [1] "age_score_mdcr and age_score_mimiciv are not identical"
```

The differences in the age score is simple. In the MIMIC-IV Code, the records without an age were dropped, `medicalcoder::comorbidities()` returned rows for these subjects. In all cases there where no comorbidities flagged.

```
mdcr_v_mimiciv[, .N, keyby = .(age_score_mdcr, age_score_mimiciv)]
## Key: <age_score_mdcr, age_score_mimiciv>
```

```
##    age_score_mdcr age_score_mimiciv      N
##             <int>             <int>  <int>
## 1:             NA                 0 141175
## 2:              0                 0 164387
## 3:              1                 1  93943
## 4:              2                 2 108202
## 5:              3                 3  91270
## 6:              4                 4  88226

medicalcoder_charlson_current[
  is.na(age_score), all(cci == 0) & all(cmrb_flag == 0)
]
## [1] TRUE

mdcr_v_mimiciv[, age_score_mdcr := NULL]
mdcr_v_mimiciv[, age_score_mimiciv := NULL]
```

All that is left in `mdcr_v_mimiciv` are the id.vars and the `num_cmrb`, and `cmrb_flag`. These columns are from `medicalcoder::comorbidities()` and report the number of comorbidities flagged and indicator for any comorbidity.

```
head(mdcr_v_mimiciv)
## Key: <subject_id, enc_seq>
##    subject_id enc_seq num_cmrb cmrb_flag  hadm_id
##         <int>   <int>    <int>     <int>    <int>
## 1:   10000032       1        2         1 22595853
## 2:   10000032       2        2         1 22841357
## 3:   10000032       3        2         1 29079034
## 4:   10000032       4        2         1 25742920
## 5:   10000048       1        0         0       NA
## 6:   10000058       1        0         0       NA
```

3.3.2 `medicalcoder::comorbidities()` vs `comorbidity::comorbidity()`

We will compare the returns from the two utilities for both Charlson and Elixhauser comorbidities and the flagging of the comorbidities on an encounter by encounter basis (`flag.method = 'current'`) and longitudinally (`flag.method = 'cumulative'`).

3.3.2.1 Charlson

```r
mdcr_v_cmrb_charlson_current <-
  merge(
    x = medicalcoder_charlson_current,
    y = comorbidity_charlson_current,
    all = TRUE,
    by = c("subject_id", "enc_seq"),
    suffixes = c("_mdcr", "_cmrb")
  )

mdcr_v_cmrb_charlson_cumulative <-
  merge(
    x = medicalcoder_charlson_cumulative,
    y = comorbidity_charlson_cumulative,
    all = TRUE,
    by = c("subject_id", "enc_seq"),
    suffixes = c("_mdcr", "_cmrb")
  )
```

The following maps the columns in the returned objects.

```r
mvc_columns <-
  data.table::fread(text = "
    medicalcoder   | comorbidity
    aidshiv        | aids
    cebvd          | cevd
    chf_mdcr       | chf_cmrb
    hp_mdcr        | hp_cmrb
    copd           | cpd
    dem            | dementia
    dm             | diab
    dmc            | diabwc
    mal            | canc
    mi_mdcr        | mi_cmrb
    mld_mdcr       | mld_cmrb
    msld_mdcr      | msld_cmrb
    mst            | metacanc
    pud_mdcr       | pud_cmrb
    pvd_mdcr       | pvd_cmrb
    rhd            | rheumd
    rnd            | rend
    cci            | score
  ")
```

The following `for` loop will check if the columns are identical, removing from the object is so, and printing a message if not.

```
for (i in seq_len(nrow(mvc_columns))) {
  jm <- mvc_columns[["medicalcoder"]][i]
  jc <- mvc_columns[["comorbidity"]][i]
  e <- identical(
    mdcr_v_cmrb_charlson_current[[jm]],
    mdcr_v_cmrb_charlson_current[[jc]]
  )
  if (e) {
    mdcr_v_cmrb_charlson_current[[jm]] <- NULL
    mdcr_v_cmrb_charlson_current[[jc]] <- NULL
  } else {
    print(sprintf("mdcr_v_cmrb_charlson_current: %s is not identical to %s", jm, jc))
  }
  e <- identical(mdcr_v_cmrb_charlson_cumulative[[jm]], mdcr_v_cmrb_charlson_cumulative[[jc]]
  if (e) {
    mdcr_v_cmrb_charlson_cumulative[[jm]] <- NULL
    mdcr_v_cmrb_charlson_cumulative[[jc]] <- NULL
  } else {
    print(sprintf("mdcr_v_cmrb_charlson_cumulative: %s is not identical to %s", jm, jc))
  }
}
## [1] "mdcr_v_cmrb_charlson_current: mal is not identical to canc"
## [1] "mdcr_v_cmrb_charlson_cumulative: mal is not identical to canc"
## [1] "mdcr_v_cmrb_charlson_current: cci is not identical to score"
## [1] "mdcr_v_cmrb_charlson_cumulative: cci is not identical to score"
```

There is a mismatch for malignant cancer. In all cases `medicalcoder::comorbidities()` flagged malignant cancer while `comorbidity::comorbidity()` did not flag.

```
mdcr_v_cmrb_charlson_current[mal != canc, all(mal > canc)]
## [1] TRUE
```

Let's look for the ICD-codes that `medicalcoder::comorbidities()` flagged and `comorbidity::comorbidity()` did not.

```
DT <-
  mimicivDT[
    mdcr_v_cmrb_charlson_current[mal != canc, .(cid)],
    on = "cid"
```

```
  ][
    ,
    code_id := as.numeric(factor(icd_code))
  ]

medicalcoder::comorbidities(
  data = DT,
  id.var = c("icd_code", "code_id"),
  icd.codes = "icd_code",
  icdv.var = "icd_version",
  dx.var = "dx",
  poa = 1L,
  primarydx = 0L,
  method = "charlson_quan2005"
)[mal == 1, .(icd_code, code_id, mal)]
##    icd_code code_id   mal
##      <char>   <num> <int>
## 1:   C4A111      48     1
## 2:    C4A39      49     1
## 3:    C4A52      50     1
## 4:    C4A62      51     1
## 5:    C4A71      52     1
## 6:    C4A72      53     1
## 7:     C4A9      54     1

comorbidity::comorbidity(
  x = DT,
  id = "code_id",
  code = "icd_code",
  map = "charlson_icd10_quan",
  assign0 = TRUE
  ) |>
subset(subset = canc == 1, select = c("code_id", "canc"))
## [1] code_id canc
## <0 rows> (or 0-length row.names)
```

The ICD-10-CM codes C4A.x are flagged as a malignancy by `medicalcoder::comorbidities()` but not flagged by `comorbidity::comorbidity()`. Table 2 of Quan et al. (2005) lists C45.x - C58.x as mapping to any malignancy. We argue that C4A.x is included in that range of codes and that these are false negatives from `comorbidity::comorbidity()`.

NOTE: *medicalcoder* has a robust internal database of ICD codes. End users can access the

codes via `medicalcoder::get_icd_codes()`.

```
subset(
  medicalcoder::get_icd_codes(with.description = TRUE),
  code %in% c("C4A111", "C4A39", "C4A52", "C4A62", "C4A71", "C4A72", "C4A9")
  )
##        icdv dx full_code   code src known_start known_end assignable_start
## 126898   10  1   C4A.111 C4A111 cms        2019      2026             2019
## 126910   10  1    C4A.39  C4A39 cms        2014      2026             2014
## 126914   10  1    C4A.52  C4A52 cms        2014      2026             2014
## 126919   10  1    C4A.62  C4A62 cms        2014      2026             2014
## 126922   10  1    C4A.71  C4A71 cms        2014      2026             2014
## 126923   10  1    C4A.72  C4A72 cms        2014      2026             2014
## 126925   10  1     C4A.9   C4A9 cms        2014      2026             2014
##        assignable_end
## 126898           2026
## 126910           2026
## 126914           2026
## 126919           2026
## 126922           2026
## 126923           2026
## 126925           2026
##                                                                      desc
## 126898 Merkel cell carcinoma of right upper eyelid, including canthus
## 126910                   Merkel cell carcinoma of other parts of face
## 126914                        Merkel cell carcinoma of skin of breast
## 126919   Merkel cell carcinoma of left upper limb, including shoulder
## 126922       Merkel cell carcinoma of right lower limb, including hip
## 126923        Merkel cell carcinoma of left lower limb, including hip
## 126925                             Merkel cell carcinoma, unspecified
##        desc_start desc_end
## 126898       2019     2026
## 126910       2014     2026
## 126914       2014     2026
## 126919       2014     2026
## 126922       2014     2026
## 126923       2014     2026
## 126925       2014     2026
```

3.3.2.2 Elixhauser

The checks between `medicalcoder::comorbidities()` and `comorbidity::comorbidity()`

for Elixhauser comorbidities is done the same way the prior checks have been done.

```
mdcr_v_cmrb_elixhauser_current <-
  merge(
    x = medicalcoder_elixhauser_current,
    y = comorbidity_elixhauser_current,
    all = TRUE,
    by = c("subject_id", "enc_seq"),
    suffixes = c("_mdcr", "_cmrb")
  )

mdcr_v_cmrb_elixhauser_cumulative <-
  merge(
    x = medicalcoder_elixhauser_cumulative,
    y = comorbidity_elixhauser_cumulative,
    all = TRUE,
    by = c("subject_id", "enc_seq"),
    suffixes = c("_mdcr", "_cmrb")
  )
```

```
mvc_columns <-
  data.table::fread(text = "
    medicalcoder        | comorbidity
    AIDS                | aids
    ALCOHOL             | alcohol
    ANEMDEF             | dane
    ARTH                | rheumd
    BLDLOSS             | blane
    CARDIAC_ARRHYTHMIAS | carit
    CHF                 | chf
    CHRNLUNG            | cpd
    COAG                | coag
    DEPRESS             | depre
    DM                  | diabunc
    DMCX                | diabc
    DRUG                | drug
    HTN_CX              | hypc
    HTN_UNCX            | hypunc
    HYPOTHY             | hypothy
    LIVER               | ld
    LYMPH               | lymph
    LYTES               | fed
    METS                | metacanc
```

```
    NEURO                | ond
    OBESE                | obes
    PARA                 | para
    PERIVASC             | pvd
    PSYCH                | psycho
    PULMCIRC             | pcd
    RENLFAIL             | rf
    TUMOR                | solidtum
    ULCER                | pud
    VALVE                | valv
    WGHTLOSS             | wloss
  ")
```

```
for (i in seq_len(nrow(mvc_columns))) {
  jm <- mvc_columns[["medicalcoder"]][i]
  jc <- mvc_columns[["comorbidity"]][i]
  e <- identical(
    mdcr_v_cmrb_elixhauser_current[[jm]],
    mdcr_v_cmrb_elixhauser_current[[jc]]
  )
  if (e) {
    mdcr_v_cmrb_elixhauser_current[[jm]] <- NULL
    mdcr_v_cmrb_elixhauser_current[[jc]] <- NULL
  } else {
    print(sprintf("mdcr_v_cmrb_elixhauser_current: %s is not identical to %s", jm, jc))
  }
  e <- identical(
    mdcr_v_cmrb_elixhauser_cumulative[[jm]],
    mdcr_v_cmrb_elixhauser_cumulative[[jc]]
  )
  if (e) {
    mdcr_v_cmrb_elixhauser_cumulative[[jm]] <- NULL
    mdcr_v_cmrb_elixhauser_cumulative[[jc]] <- NULL
  } else {
    print(sprintf("mdcr_v_cmrb_elixhauser_cumulative: %s is not identical to %s", jm, jc))
  }
}
```

The remaining columns in `mdcr_v_cmrb_elixhauser_current` and `mdcr_v_cmrb_elixhauser_cumulative` are unique to `medicalcoder::comorbidities()`. `HTN_C` is any hypertension.

```
mdcr_v_cmrb_elixhauser_current
## Key: <subject_id, enc_seq>
##         subject_id enc_seq HTN_C num_cmrb cmrb_flag mortality_index
##              <int>   <int> <int>    <int>     <int>           <int>
##      1:   10000032       1     0        3         1               2
##      2:   10000032       2     0        4         1              29
##      3:   10000032       3     0        4         1              27
##      4:   10000032       4     0        3         1              18
##      5:   10000048       1     0        0         0               0
##     ---
## 687199:   19999829       1     0        0         0               0
## 687200:   19999840       1     1        2         1               4
## 687201:   19999840       2     1        2         1               4
## 687202:   19999914       1     0        0         0               0
## 687203:   19999987       1     0        3         1              11
##         readmission_index         cid
##                     <int>      <char>
##      1:                22 10000032__1
##      2:                33 10000032__2
##      3:                36 10000032__3
##      4:                26 10000032__4
##      5:                 0 10000048__1
##     ---
## 687199:                 0 19999829__1
## 687200:                 6 19999840__1
## 687201:                 6 19999840__2
## 687202:                 0 19999914__1
## 687203:                23 19999987__1
mdcr_v_cmrb_elixhauser_cumulative
## Key: <subject_id, enc_seq>
##         subject_id enc_seq HTN_C num_cmrb cmrb_flag mortality_index
##              <int>   <int> <int>    <int>     <int>           <int>
##      1:   10000032       1     0        3         1               2
##      2:   10000032       2     0        5         1              24
##      3:   10000032       3     0        6         1              33
##      4:   10000032       4     0        6         1              33
##      5:   10000048       1     0        0         0               0
##     ---
## 687199:   19999829       1     0        0         0               0
## 687200:   19999840       1     1        2         1               4
## 687201:   19999840       2     1        2         1               4
## 687202:   19999914       1     0        0         0               0
```

```
## 687203:   19999987       1     0        3         1              11
##         readmission_index         cid
##                     <int>      <char>
##      1:                22 10000032__1
##      2:                37 10000032__2
##      3:                47 10000032__3
##      4:                47 10000032__4
##      5:                 0 10000048__1
##     ---
## 687199:                 0 19999829__1
## 687200:                 6 19999840__1
## 687201:                 6 19999840__2
## 687202:                 0 19999914__1
## 687203:                23 19999987__1
```

```
stopifnot(
  identical(
    medicalcoder_elixhauser_current[["HTN_C"]],
    as.integer(
      medicalcoder_elixhauser_current[["HTN_UNCX"]] |
        medicalcoder_elixhauser_current[["HTN_CX"]]
    )
  ),
  identical(
    medicalcoder_elixhauser_cumulative[["HTN_C"]],
    as.integer(
      medicalcoder_elixhauser_cumulative[["HTN_UNCX"]] |
        medicalcoder_elixhauser_cumulative[["HTN_CX"]]
    )
  )
)
```

The `mortality_index` and `readmission_index` are based on the weights from AHRQ in this case were `method = "elixhauser_quan2005"`. For `method = "elixhauser_ahrqXXXX"` other weights may be applied. To see the specific weights used by method call `medicalcoder::get_elixhauser_index_scores()`

```
str(
  medicalcoder::get_elixhauser_index_scores()
)
## 'data.frame':    112 obs. of  11 variables:
##  $ condition                : chr  "AIDS" "AIDS" "ALCOHOL" "ALCOHOL" ...
```

```
##  $ index                     : chr  "mortality" "readmission" "mortality" "readmission" ..
##  $ elixhauser_ahrq_web       : int  0 19 -1 6 -2 9 0 4 NA NA ...
##  $ elixhauser_elixhauser1988: int  0 19 -1 6 -2 9 0 4 NA NA ...
##  $ elixhauser_quan2005       : int  0 19 -1 6 -2 9 0 4 NA NA ...
##  $ elixhauser_ahrq2022       : int  -4 5 -1 3 -3 5 NA NA -1 2 ...
##  $ elixhauser_ahrq2023       : int  -4 5 -1 3 -3 5 NA NA 0 2 ...
##  $ elixhauser_ahrq2024       : int  -4 5 -1 3 -3 5 NA NA 0 2 ...
##  $ elixhauser_ahrq2025       : int  -4 5 -1 3 -3 5 NA NA 0 2 ...
##  $ elixhauser_ahrq2026       : int  -4 5 -1 3 -3 5 NA NA 0 2 ...
##  $ elixhauser_ahrq_icd10     : int  -4 5 -1 3 -3 5 NA NA 0 2 ...
```

3.3.3 `medicalcoder::comorbidities()` vs `pccc::ccc()`

```
mdcr_v_pccc_current <-
  merge(
    x = medicalcoder_pcccv2.0_current,
    y = pccc_pcccv2.0_current,
    all = TRUE,
    by = c("subject_id", "enc_seq"),
    suffixes = c("_mdcr", "_pccc")
  )

mdcr_v_pccc_cumulative <-
  merge(
    x = medicalcoder_pcccv2.0_cumulative,
    y = pccc_pcccv2.0_cumulative,
    all = TRUE,
    by = c("subject_id", "enc_seq"),
    suffixes = c("_mdcr", "_pccc")
  )
```

The return from `pccc::ccc()` does not include all the `subject_id` and `enc_seq`. The missing ones are all flagged as no comorbidities by `medicalcoder::comorbidities()`. Implicitly this is the same result.

```
# verify that medicalcoder's flag for any comorbidity (cmrb_flag) is zero when
# the ccc_flag from pccc::ccc()
mdcr_v_pccc_current[is.na(ccc_flag), all(cmrb_flag == 0)]
## [1] TRUE
```

```
mdcr_v_pccc_cumulative[is.na(ccc_flag), all(cmrb_flag == 0)]
## [1] TRUE
```

```
for (j in c(unique(medicalcoder::get_pccc_conditions()$condition), "tech_dep", "transplant",
  if (j %in% c("tech_dep", "transplant")) {
    jm <- sprintf("any_%s", j)
    jp <- j
  } else if (j == "ccc_flag") {
    jm <- "cmrb_flag"
    jp <- j
  } else {
    jm <- sprintf("%s_mdcr", j)
    jp <- sprintf("%s_pccc", j)
  }
  e1 <- identical(mdcr_v_pccc_current[[jm]], mdcr_v_pccc_current[[jp]])
  e2 <- identical(mdcr_v_pccc_cumulative[[jm]], mdcr_v_pccc_cumulative[[jp]])
  if (e1) {
    mdcr_v_pccc_current[[jm]] <- NULL
    mdcr_v_pccc_current[[jp]] <- NULL
  } else {
    print(sprintf("for mdcr_v_pccc_current: %s is not identical to %s", jm, jp))
  }
  if (e1) {
    mdcr_v_pccc_cumulative[[jm]] <- NULL
    mdcr_v_pccc_cumulative[[jp]] <- NULL
  } else {
    print(sprintf("for mdcr_v_pccc_cumulative: %s is not identical to %s", jm, jp))
  }
}
## [1] "for mdcr_v_pccc_current: any_tech_dep is not identical to tech_dep"
## [1] "for mdcr_v_pccc_cumulative: any_tech_dep is not identical to tech_dep"
## [1] "for mdcr_v_pccc_current: any_transplant is not identical to transplant"
## [1] "for mdcr_v_pccc_cumulative: any_transplant is not identical to transplant"
```

There are some differences between the results for technology dependence (`tech_dep`) and transplant. The underlying reason for these differences is that the implementation of `medicalcoder::comorbidities()` allows for flagging of subconditions. Under this paradigm both technology dependence and transplant are subconditions. There are several ICD codes which had/have errors in *pccc* version 1.0.7.

```
DT <-
  mimicivDT[
      mdcr_v_pccc_current[
        any_tech_dep != tech_dep | any_transplant != transplant
      ],
      on = c("cid", "subject_id", "enc_seq")
    ][
      , .(icd_code, icd_version, dx)
    ][
      ,
      code_id := paste0(
        "ICD-", icd_version, "-", ifelse(dx == 1, "CM", "PCS"), " ", icd_code
      )
    ] |>
  unique()

mdcr <-
  medicalcoder::comorbidities(
    data = DT,
    id.var = "code_id",
    icd.codes = "icd_code",
    icdv.var = "icd_version",
    dx.var = "dx",
    poa = 1L,
    method = 'pccc_v2.0'
  )

DT_for_pccc <-
  split(DT, by = "icd_version") |>
  lapply(
    data.table::dcast,
    formula = code_id ~ ifelse(dx == 1, "DX", "PR"),
    value.var = "icd_code"
    )

pccc <-
  rbind(
    pccc::ccc(
      data = DT_for_pccc[["9"]],
      id = code_id,
      dx_cols = grep("^DX", names(DT_for_pccc[["9"]]), value = TRUE),
      pc_cols = grep("^PR", names(DT_for_pccc[["9"]]), value = TRUE),
```

```
      icdv = 9
    ),
    pccc::ccc(
      data = DT_for_pccc[["10"]],
      id = code_id,
      dx_cols = grep("^DX", names(DT_for_pccc[["10"]]), value = TRUE),
      pc_cols = grep("^PR", names(DT_for_pccc[["10"]]), value = TRUE),
      icdv = 10
    )
  )

delta <- merge(mdcr, pccc, all = TRUE, by = "code_id")

delta <-
  delta[
    any_tech_dep != tech_dep | any_transplant != transplant,
    .(code_id, any_tech_dep, any_transplant, tech_dep, transplant)
  ]

delta[, .N, keyby = .(any_tech_dep, tech_dep, any_transplant, transplant)]
## Key: <any_tech_dep, tech_dep, any_transplant, transplant>
##    any_tech_dep tech_dep any_transplant transplant     N
##           <int>    <int>          <int>      <int> <int>
## 1:            0        0              1          0     7
## 2:            1        0              0          0     9
## 3:            1        0              1          1     2
## 4:            1        1              1          0     1

data.table::setkey(delta, code_id)

print(delta, nrow = Inf)
## Key: <code_id>
##             code_id any_tech_dep any_transplant tech_dep transplant
##              <char>        <int>          <int>    <int>      <int>
##  1: ICD-10-CM Z4901            1              0        0          0
##  2: ICD-10-CM Z4931            1              0        0          0
##  3:  ICD-10-CM Z941            0              1        0          0
##  4:  ICD-10-CM Z942            1              1        1          0
##  5:  ICD-10-CM Z944            0              1        0          0
##  6: ICD-10-CM Z9481            0              1        0          0
##  7: ICD-10-CM Z9482            0              1        0          0
##  8: ICD-10-CM Z9483            0              1        0          0
```

```
##  9: ICD-10-CM Z9484           0           1           0           0
## 10:   ICD-9-CM 3491           1           0           0           0
## 11:   ICD-9-CM V420           0           1           0           0
## 12:  ICD-9-CM V4585           1           1           0           1
## 13:  ICD-9-CM V5391           1           1           0           1
## 14:   ICD-9-CM V560           1           0           0           0
## 15:   ICD-9-CM V561           1           0           0           0
## 16:   ICD-9-CM V562           1           0           0           0
## 17:   ICD-9-CM V568           1           0           0           0
## 18:  ICD-9-CM V6546           1           0           0           0
## 19:  ICD-9-PCS 8606           1           0           0           0
```

These are known and reported issues:

- ICD-10-CM Z94x
  - GitHub Issue: https://github.com/CUD2V/pccc/issues/56
  - Specifically Z94.1, Z94.2, Z94.4, Z94.81, Z94.82, Z94.83, Z94.84 All documented as subcondition transplant but are missing from the transplant set in the pccc_1.0.7/src/pccc.cpp
- ICD-9-CM 3491
  - GitHub Issue: https://github.com/CUD2V/pccc/issues/45
  - Documented and implemented for neuromusc in `pccc::ccc()` tech_dep is missing
- ICD-9-CM V420
  - GitHub Issue: https://github.com/CUD2V/pccc/issues/54
  - Documented as renal (transplant); missing from the transplant set in the pccc_1.0.7/src/pccc.cpp
- ICD-9-CM V4585
  - GitHub Issue: https://github.com/CUD2V/pccc/issues/49
  - Documented for version 2 as metabolic (devices) but is only listed under metabolic in the software. Because of how `medicalcoder::comorbidities()` is implemented the subconditon (devices) had to be provided in the tables for `method = "pccc_v2.0"`. This results in the (improved) mapping to technology dependence. Perhaps more concerning is that the mapping for V45.85 in `pccc::ccc()` maps to a transplant, which is incorrect. GitHub issue and pull request have been submitted to resolve this issue.
- ICD-9-CM V5391
  - GitHub Issue: https://github.com/CUD2V/pccc/issues/51

  - In `pccc::ccc()` this codes maps to transplant when it should map to technology dependence.

- ICD-9-CM V56.x
  - GitHub Issue: https://github.com/CUD2V/pccc/issues/46
  - Same as ICD-9-CM 349.1

- ICD-9-CM V6546
  - GitHub Issue: https://github.com/CUD2V/pccc/issues/52
  - Missing tech dependence flag from `pccc::ccc()`

- ICD-9-PCS 8606
  - GitHub Issue: https://github.com/CUD2V/pccc/issues/48
  - Missing tech dependence flag from `pccc::ccc()`

We remove the transplant and technology dependence columns from the `mdcr_v_pccc_current` and `mdcr_v_pccc_cumulative` objects.

```
mdcr_v_pccc_current[, any_transplant := NULL]
mdcr_v_pccc_current[, any_tech_dep := NULL]
mdcr_v_pccc_current[, transplant := NULL]
mdcr_v_pccc_current[, tech_dep := NULL]
mdcr_v_pccc_cumulative[, any_transplant := NULL]
mdcr_v_pccc_cumulative[, any_tech_dep := NULL]
mdcr_v_pccc_cumulative[, transplant := NULL]
mdcr_v_pccc_cumulative[, tech_dep := NULL]
```

The only remaining columns are the id columns and two columns from `medicalcoder::comorbidities()`: `misc`, indicating a technology dependence or transplant not classified as one of the other comorbidities, and `num_cmrb`, the number of comorbidities flagged.

```
str(mdcr_v_pccc_current)
## Classes 'medicalcoder_comorbidities', 'data.table' and 'data.frame': 687203 obs. of  5 va
##  $ subject_id: int  10000032 10000032 10000032 10000032 10000048 10000058 10000068 100000
##  $ enc_seq   : int  1 2 3 4 1 1 1 1 2 1 ...
##  $ misc      : int  0 0 1 1 0 0 0 0 0 0 ...
##  $ num_cmrb  : int  1 2 3 3 0 0 0 2 2 0 ...
##  $ cid       : chr  "10000032__1" "10000032__2" "10000032__3" "10000032__4" ...
##  - attr(*, "sorted")= chr [1:2] "subject_id" "enc_seq"
##  - attr(*, ".internal.selfref")=<externalptr>
str(mdcr_v_pccc_cumulative)
## Classes 'medicalcoder_comorbidities', 'data.table' and 'data.frame': 687203 obs. of  5 va
```

```
##  $ subject_id: int  10000032 10000032 10000032 10000032 10000048 10000058 10000068 100000
##  $ enc_seq   : int  1 2 3 4 1 1 1 1 2 1 ...
##  $ misc      : int  0 0 1 1 0 0 0 0 0 0 ...
##  $ num_cmrb  : int  1 2 3 3 0 0 0 2 2 0 ...
##  $ cid       : chr  "10000032__1" "10000032__2" "10000032__3" "10000032__4" ...
##  - attr(*, "sorted")= chr [1:2] "subject_id" "enc_seq"
##  - attr(*, ".internal.selfref")=<externalptr>
```

3.4 Current vs Cumulative Flagging

3.4.1 Reduced Severity

Let's look at the impact of longitudinal flagging vs encounter level flagging of Charlson comorbidities.

Figure 1 shows the number of encounters flagged with a given condition for `flag.method = 'current'` and `flag.method = 'cumulative'` for the Charlson condition flagged by `medicalcoder::comorbidities()`.

Diabetes is a great example for the benefits of `flag.method = "cumulative"`. As we account for a patient history, the number of encounters flagged as having uncomplicated diabetes decreases and the number of encounters flagged as having complicated diabetes increases.

An example of a patient record where on an encounter level flagging approach diabetes will be missed entirely and the severity is inaccurate is in Table 3. Subject 10009326 had 17 encounters. At least one ICD code mapping to uncomplicated diabetes under the Charlson *a la* Quan et al. (2005) appeared in 12 encounters, and 1 encounter for complicated diabetes. Under an encounter-level only flagging of comorbidities, this patient would erroneously not have diabetes of any kind on 4 encounters. Using the longitudinal flagging in `medicalcoder::comorbidities()` via `flag.method = 'cumulative'`, we have diabetes flagged on all encounters, and the severity correctly marked for encounters later in the patient record.

For Elixhauser, Figure 2, shows similar behavior for diabetes as Charlson.

3.4.2 PCCC

Figure 3 and Figure 4 illustrate the impact of the flagging method when identifying PCCC comorbidities under PCCC v2 and PCCC v3 respectively. Figure 4 is of particular interest as the implications of technology-dependent codes in PCCC v3 are handled have considerable impact on the flagged comorbidities.

Let's look at the records for `subject_id = 10728333`. Table 4 shows the flagging of metabolic, miscellaneous, and respiratory conditions for PCCC v3 using `flag.method = 'current'` or

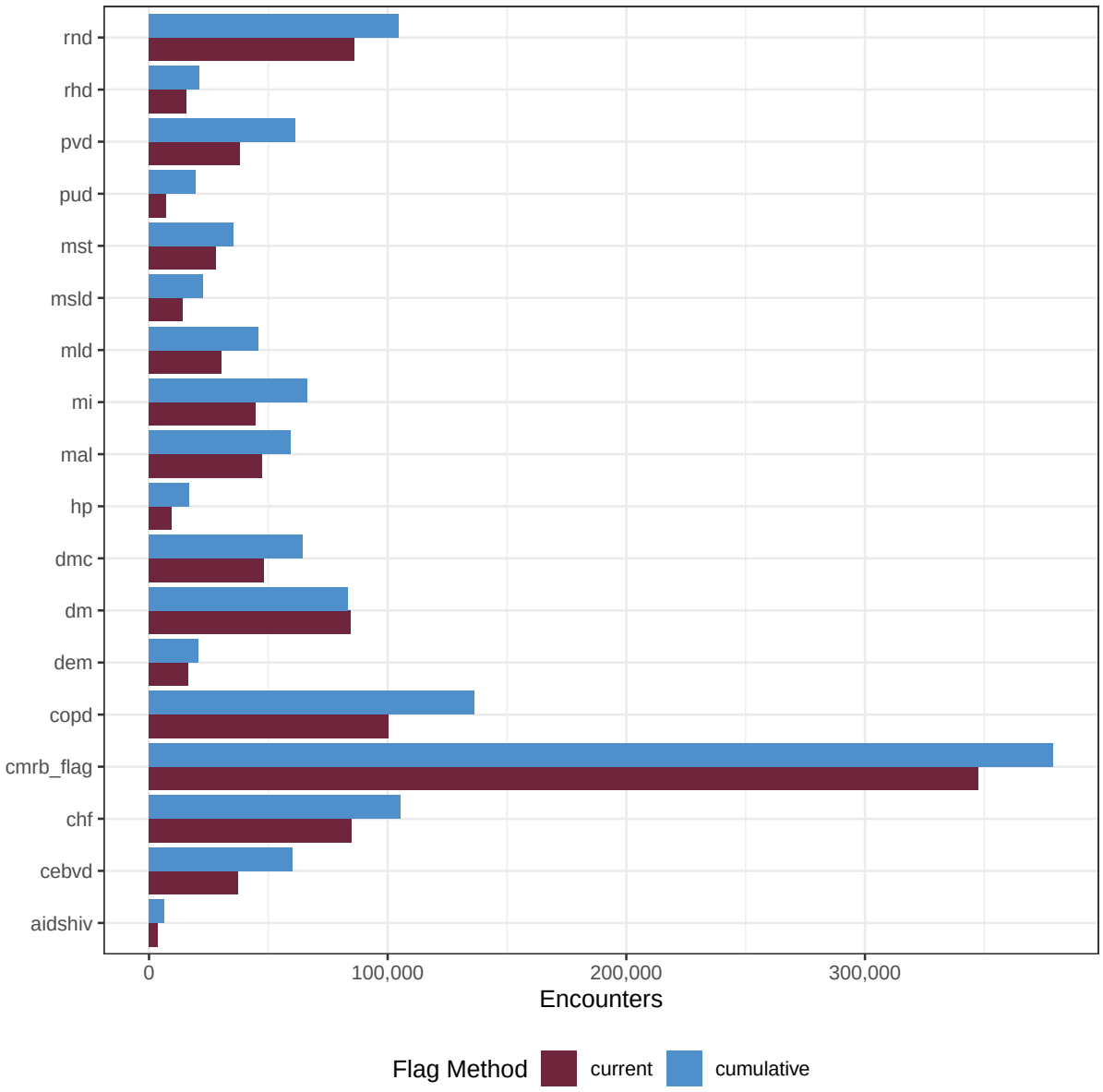

Figure 1: Number of encounters flagged with a Charlson comorbidities based on encounter only (`flag.method = "current"`) verses the whole patient history (`flag.method = "cumulative"`).

Table 3: Flagging Diabetes under Charlson comorbidities for MIMIC-IV subject_id 10009326 using `medicalcoder::comorbidities()` `flag.method = "current"` and `flag.method = "cumulative"`.

	Current		Cumulative	
Encounter	Complicated	Uncomplicated	Complicated	Uncomplicated
1		X		X
2				X
3		X		X
4		X		X
5				X
6		X		X
7		X		X
8		X		X
9				X
10		X		X
11				X
12	X		X	
13		X	X	
14		X	X	
15		X	X	
16		X	X	
17		X	X	

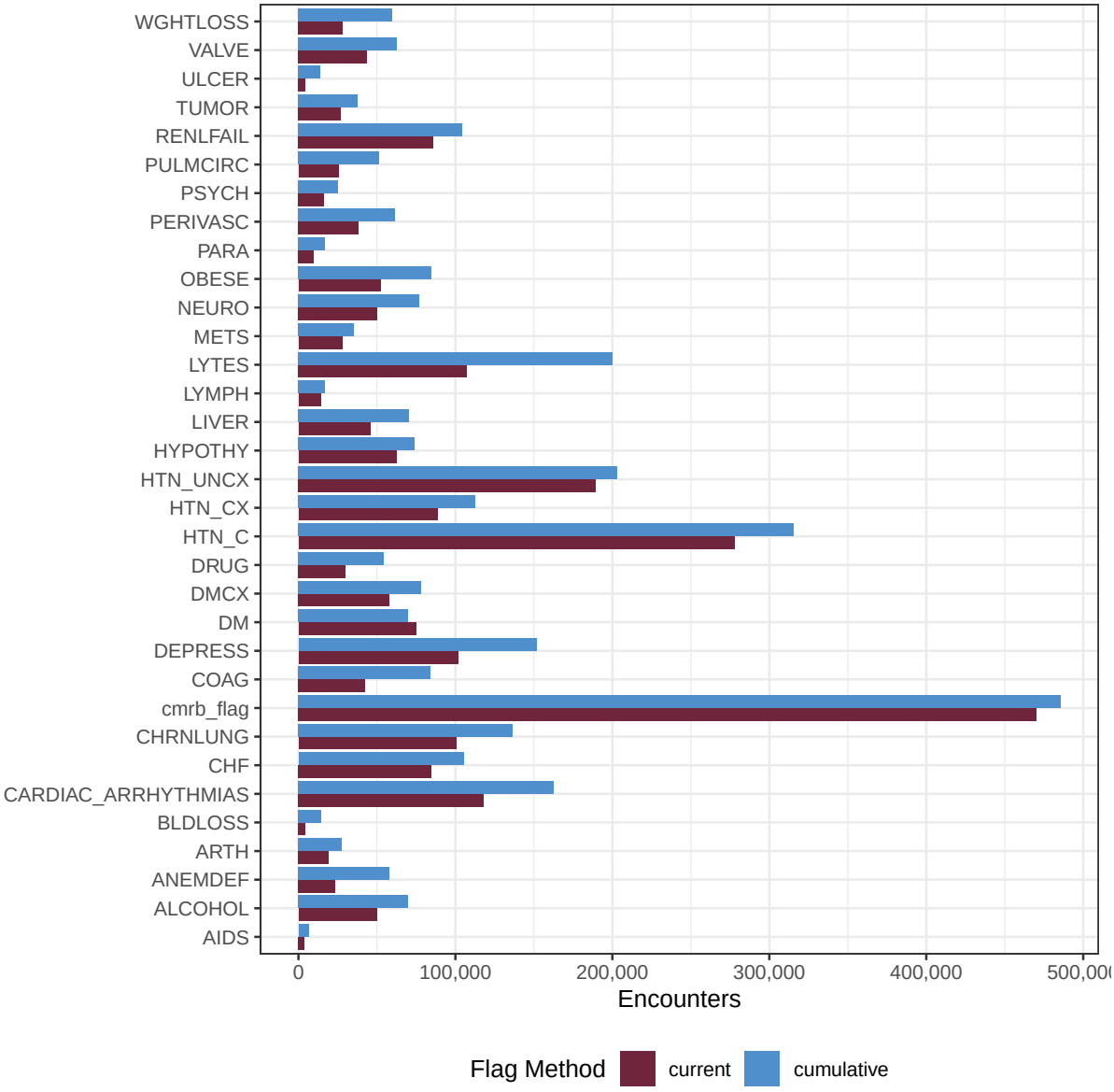

Figure 2: Number of encounters flagged with a Elixhauser comorbidities based on encounter only (`flag.method = current`) verses the whole patient history (`flag.method = "cumulative"`).

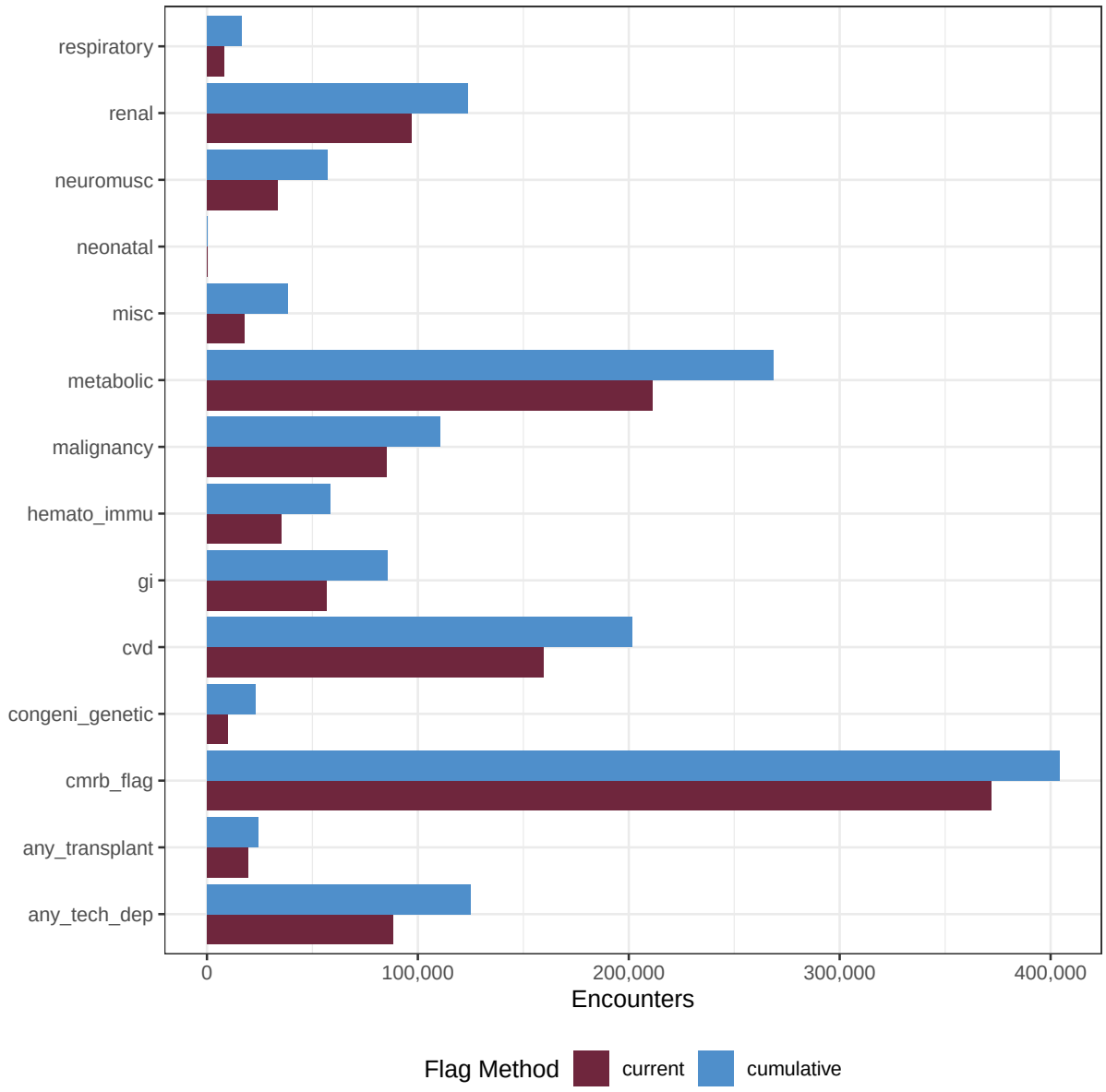

Figure 3: Number of encounters flagged with a PCCC v2 comorbidities based on encounter only (`flag.method = current`) verses the whole patient history (`flag.method = "cumulative"`).

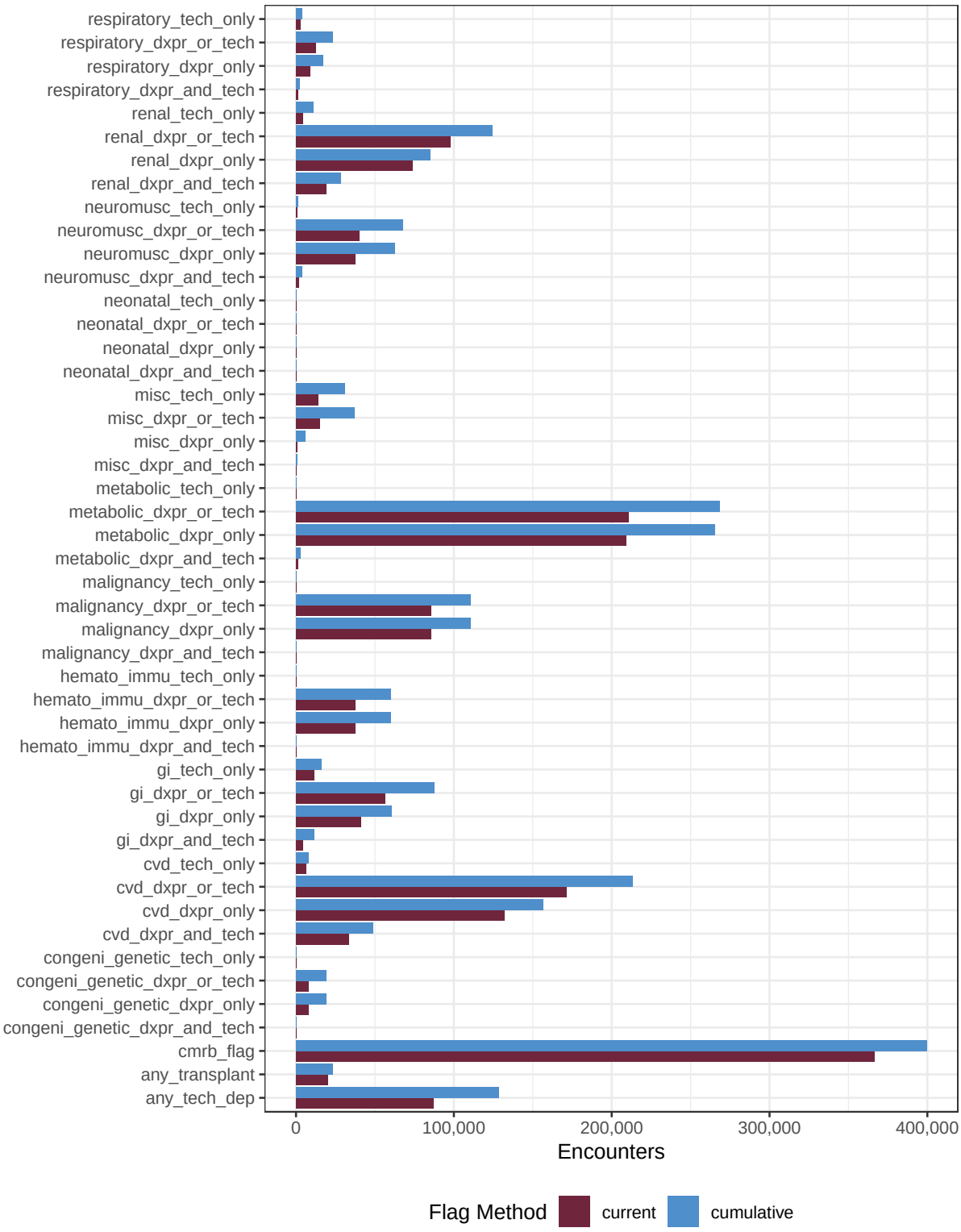

Figure 4: Number of encounters flagged with a PCCC v3 comorbidities based on encounter only (`flag.method = current`) verses the whole patient history (`flag.method = "cumulative"`).

`flag.method = 'cumulative'`. For this patient, on encounter 1, a technology-dependent ICD code related to a respiratory comorbidity was observed in the patient record. No other ICD codes related to a PCCC v3 comorbidity were observed for encounter 1. As such, no comorbidities are flagged for encounter 1 as at least one non-technology-dependent ICD code is required for any comorbidity to be flagged. Encounter 2 had no PCCC v3 related ICD codes reported. Encounter 3 had a technology-dependent ICD code for a miscellaneous comorbidity reported and again, because no non-technology-dependent ICD was reported no comorbidity is flagged. Starting on encounter 4, a non-technology-dependent ICD code for a metabolic comorbidity was reported. Under the `flag.method = 'current'` that is the only code that is reported and thus only one comorbidity is reported. Under `flag.method = 'cumulative'` the technology-dependent ICD codes from encounters 1 and 3 have been carried forward in the record to encounter 4 and are thus flagged. If the end user were to try to build the cumulative comorbidities from the results of the `flag.method = 'current'` results, the end user would never capture the miscellaneous comorbidity, and would fail to capture the respiratory comorbidity until encounter 6, and would incorrectly classify as non-technology-dependent until the final encounter.

3.4.3 AHRQ Example

AHRQ Elixhauser for years 2022 and beyond use ICD-10 diagnostic codes and require some ICD-codes to be present-on-admission for the comorbidity to be flag. The MIMIC data does not have a present-on-admission flag. For this example let's assume that all codes are not POA.

```
common_args <-
  list(
    data = mimicivDT,
    id.vars = c("subject_id", "enc_seq"),
    icd.codes = "icd_code",
    icdv.var = "icd_version",
    dx.var = "dx",
    poa = 0L,
    primarydx = 0L,
    method = "elixhauser_ahrq_icd10"
  )

flgcurrent <-
  do.call(
    medicalcoder::comorbidities,
    c(common_args, list(flag.method = "current"))
  )

flgcumulative <-
```

Table 4

Table 5: Flagging of comorbidities under PCCC v3 for MIMIC-IV subject 10728333. Cells marked with DxPr denote that the comorbidity was flagged due to a ICD code that is not technology-dependent. Tech denotes flagging of a comorbidity base on the presence of at least one technology-dependent ICD code and the presence of at least one non-technology-dependent code for another comorbidity. DxPrTech denotes flagging due to both technology-dependent and non-technology-dependent ICD codes. Under the Any Tech columns we mark the encounters where some tech dependence is identified. For this patient, on encounter 1, a technology-dependent ICD code for a respiratory comorbidity was found in the subject's record. Because no non-technology-dependent code was observed on encounter 1, under PCCC v3 this subject has no comorbidities for encounter 1. This persists for encounters 2 and 3 with respect to respiratory and on encounter 3 for miscellaneous as well. On encounter 4 a non-technology-dependent code for a metabolic comorbidity was reported. Under a cumulative flagging paradigm the miscellaneous and respiratory comorbidities are also flagged as the reported codes from encounters 3 and 1 have been carried forward respectively. Under the 'current' flagging method miscellaneous is never flagged as the technology-dependent code never occurs on the same encounter as the non-technology-dependent code.

	Metabolic		Miscellaneous		Respiratory		Any Tech		Number of Comorbiditys	
Encounter	Current	Cumulative	Current	Cumulative	Current	Cumulative	Current	Cumulative	Current	Cumulative
1					*	*			0	0
2						†			0	0
3			*	*		†			0	0
4	DxPr	DxPr		Tech		Tech		1	1	3
5	DxPr	DxPr		Tech		Tech		1	1	3
6	DxPr	DxPr		Tech	DxPr	DxPrTech		1	2	3
7	DxPr	DxPr		Tech		DxPrTech		1	1	3
8	DxPr	DxPr		Tech		DxPrTech		1	1	3
9	DxPr	DxPr		Tech		DxPrTech		1	1	3
10	DxPr	DxPr		Tech		DxPrTech		1	1	3
11		DxPr		Tech		DxPrTech		1	0	3
12		DxPr		Tech		DxPrTech		1	0	3
13	DxPr	DxPr		Tech	Tech	DxPrTech	1	1	2	3

* A technology-dependent code was reported on this encounter. Since no non-technology-dependent was reported no comorbidities are flagged.
† A technology-dependent code has been carried forward in the record. Since no non-technology-dependent was reported on this, or a prior, enconter, no comorbidities are flagged.

```
  do.call(
    medicalcoder::comorbidities,
    c(common_args, list(flag.method = "cumulative"))
  )
```

Let's look at subject_id 19997538. This subject has ICD code K76.0 reported on encounter 1.

```
mimicivDT[subject_id == 19997538 & icd_code == "K760"]
## Key: <subject_id, admittime, hadm_id>
##    subject_id  hadm_id    dx seq_num icd_code icd_version chartdate      age
##         <int>    <int> <int>   <int>   <char>       <int>    <IDat>    <num>
## 1:   19997538 22701415     1      15     K760          10      <NA> 53.33048
##     admittime enc_seq         cid
##        <POSc>   <int>      <char>
## 1: 2168-05-01       1 19997538__1
```

K76.0 maps to mild liver disease and is required to be POA for the condition to be flagged.

```
subset(medicalcoder::get_icd_codes(with.description = TRUE), full_code == "K76.0")
##        icdv dx full_code code src known_start known_end assignable_start
## 151468   10  1     K76.0 K760 cdc        2001      2025             2001
## 151469   10  1     K76.0 K760 cms        2014      2026             2014
## 151470   10  1     K76.0 K760 who        2008      2019             2008
##        assignable_end                                              desc
## 151468           2025 Fatty (change of) liver, not elsewhere classified
## 151469           2026 Fatty (change of) liver, not elsewhere classified
## 151470           2019 Fatty (change of) liver, not elsewhere classified
##        desc_start desc_end
## 151468       2001     2025
## 151469       2014     2026
## 151470       2008     2019
subset(medicalcoder::get_elixhauser_codes(), full_code == "K76.0")
##      icdv dx full_code code poaexempt condition elixhauser_ahrq_web
## 8583   10  1     K76.0 K760        NA     LIVER                   0
## 8584   10  1     K76.0 K760         0 LIVER_MLD                  NA
##      elixhauser_elixhauser1988 elixhauser_quan2005 elixhauser_ahrq2022
## 8583                         0                   1                  NA
## 8584                        NA                  NA                   1
##      elixhauser_ahrq2023 elixhauser_ahrq2024 elixhauser_ahrq2025
## 8583                  NA                  NA                  NA
```

```
## 8584                    1                    1                    1
##      elixhauser_ahrq2026 elixhauser_ahrq_icd10
## 8583                  NA                   NA
## 8584                   1                    1
subset(medicalcoder::get_elixhauser_poa(), condition == "LIVER_MLD")
##    condition poa_required elixhauser_ahrq2022 elixhauser_ahrq2023
## 21 LIVER_MLD            1                    1                    1
##    elixhauser_ahrq2024 elixhauser_ahrq2025 elixhauser_ahrq2026
## 21                   1                    1                    1
##    elixhauser_ahrq_icd10
## 21                     1
```

When `flag.method = "current"` we see what we expect, mild liver disease is not flagged on any encounter. When `flag.method = "cumulative"` we see that on encounter 1 the condition is not flagged, as expected since it is not POA, but it is flagged on encounters 2 and 3 as the patient history is accounted for.

```
merge(
  x = flgcurrent,
  y = flgcumulative,
  by = c("subject_id", "enc_seq"),
  suffixes = c("_current", "_cumulative")
)[
  subject_id == 19997538,
  .SD,
  .SDcols = patterns("^(subject_id|enc_seq|LIVER_MLD_c)")
]
## Key: <subject_id, enc_seq>
##    subject_id enc_seq LIVER_MLD_current LIVER_MLD_cumulative
##         <int>   <int>             <int>                <int>
## 1:   19997538       1                 0                    0
## 2:   19997538       2                 0                    1
## 3:   19997538       3                 0                    1
```

3.5 Time Required To Compute

The times required (median and IQR) to apply the different comorbidity algorithms to the MIMIC-IV data are shown in Table 6. To process the 7,365,770 total rows of data for 364,627 unique subjects and a total of 687,203 unique hospital encounters, `medicalcoder::comorbidities()` is considerably faster than all the alternatives.

Table 6: Time, in seconds, median and IQR, to flag comorbidities via Charlson, Elixhauser, PCCC v2, and PCCC v3 and several different utilities in the MIMIC-IV v3.1 dataset. The flagging method, current or cumulative is reported.

	medicalcoder::comorbidities()	comorbidity::comorbidity()	MIMIC-IV Code	pccc::ccc()
charlson				
current	2.77 (2.70, 3.11)	54.88 (53.16, 55.71)	124.76 (122.83, 127.68)	
cumulative	6.07 (5.81, 6.38)			
elixhauser				
current	4.70 (4.14, 4.82)	85.59 (84.43, 89.50)		
cumulative	12.52 (12.17, 12.93)			
pcccv2.0				
current	2.76 (2.33, 3.21)			175.95 (172.49, 177.27)
cumulative	10.01 (9.47, 10.68)			
pcccv3.1				
current	4.47 (4.17, 5.07)			
cumulative	11.86 (11.41, 12.89)			

3.6 Summarizing Results

The *medicalcoder* package provides an S3 summary method for the objects returned by `medicalcoder::comorbidities()`. The summaries differ slightly depending of the comorbidity algorithm (`method`). Charlson and Elixhauser return lists with multiple summaries and the PCCC return is a data.frame of summaries.

When the `flag.method = 'current'`, the `conditions` element of the summary for the Charlson and Elixhauser comorbidities is a data.frame with counts and percentages of the distinct `id.vars` with a given comorbidity, and counts of comorbidities. For PCCC v2 and v3, the return is only this data.frame. When the flag method is cumulative, a warning is given stating that the summary may not be meaningful as it is an encounter level summary.

```
str(summary(medicalcoder_charlson_current), max.level = 1)
## List of 3
##  $ conditions   :'data.frame':   26 obs. of  4 variables:
##  $ age_summary  :'data.frame':   6 obs. of  3 variables:
##  $ index_summary:'data.frame':   1 obs. of  5 variables:
str(summary(medicalcoder_elixhauser_current), max.level = 1)
## List of 2
##  $ conditions   :'data.frame':   46 obs. of  3 variables:
##  $ index_summary:'data.frame':   2 obs. of  6 variables:
str(summary(medicalcoder_pcccv2.0_current), max.level = 1)
## 'data.frame':    24 obs. of  4 variables:
##  $ condition: chr  "congeni_genetic" "cvd" "gi" "hemato_immu" ...
##  $ label    : chr  "Other Congenital or Genetic Defect" "Cardiovascular" "Gastrointestina
##  $ count    : int  9960 159359 56797 35395 85183 211049 17549 39 33640 97078 ...
```

```
##  $ percent  : num  1.45 23.19 8.26 5.15 12.4 ...
str(summary(medicalcoder_pcccv3.1_current), max.level = 1)
## 'data.frame':    24 obs. of  10 variables:
##  $ condition            : chr  "congeni_genetic" "cvd" "gi" "hemato_immu" ...
##  $ label                : chr  "Other Congenital or Genetic Defect" "Cardiovascular" "Gas
##  $ dxpr_or_tech_count   : int  8156 171247 56613 37617 85384 210879 15114 39 40205 97720
##  $ dxpr_or_tech_percent : num  1.19 24.92 8.24 5.47 12.42 ...
##  $ dxpr_only_count      : int  8156 131854 41186 37617 85384 209335 896 39 37338 73916 ..
##  $ dxpr_only_percent    : num  1.19 19.19 5.99 5.47 12.42 ...
##  $ tech_only_count      : int  0 6159 11327 0 0 244 14210 0 880 4508 ...
##  $ tech_only_percent    : num  0 0.896 1.648 0 0 ...
##  $ dxpr_and_tech_count  : int  0 33234 4100 0 0 1300 8 0 1987 19296 ...
##  $ dxpr_and_tech_percent: num  0 4.836 0.597 0 0 ...

str(summary(medicalcoder_charlson_cumulative), max.level = 0)
## Warning in .charlson_summary(object): Logic for charlson summary table has been
## implemented for flag.method = 'current'.  Using this function for flag.method =
## 'cumulative' may not provide a meaningful summary.
## List of 3
```

The data.frames are provided so that end users can easily build their own summary tables using tools such as *knitr* Xie (2015) and *kableExtra* (Zhu 2024)

4 Session Info

Table 8 and Table 9 reports details on the machine, R, and the R packages used to produce this document.

References

Complex Chronic Conditions Version 3.0. 2024. Children's Hospital Association; https://www.childrenshospitals.org/content/analytics/toolkit/complex-chronic-conditions.

DeWitt, Peter, James Feinstein, and Seth Russell. 2025. *Pccc: Pediatric Complex Chronic Conditions*. https://github.com/CUD2V/pccc.

Feinstein, James A, Matt Hall, Amber Davidson, and Chris Feudtner. 2024. "Pediatric Complex Chronic Condition System Version 3." *JAMA Network Open* 7 (7): e2420579–79. https://doi.org/10.1001/jamanetworkopen.2024.20579.

Table 7: Code and resulting table for a summary of the PCCC v2 conditions flagged in the MIMIC-IV dataset.

```
kableExtra::kbl(
  x = summary(medicalcoder_pcccv2.0_current)[, 2:4],
  format = "latex",
  booktabs = TRUE,
  col.names = c("", "Encounters", "Percent"),
  digits = 2
) |>
kableExtra::kable_styling(
  latex_options = c("striped", "scale_down", "HOLD_position")
) |>
kableExtra::pack_rows("Conditions", start_row =  1, end_row = 13) |>
kableExtra::pack_rows("Total Conditions", start_row = 14, end_row = 24)
```

	Encounters	Percent
Conditions		
Other Congenital or Genetic Defect	9960	1.45
Cardiovascular	159359	23.19
Gastrointestinal	56797	8.26
Hematologic or Immunologic	35395	5.15
Malignancy	85183	12.40
Metabolic	211049	30.71
Miscellaneous, Not Elsewhere Classified	17549	2.55
Premature & Neonatal	39	0.01
Neurologic or Neuromuscular	33640	4.90
Renal Urologic	97078	14.13
Respiratory	8272	1.20
Any Technology Dependence	88353	12.86
Any Transplantation	19651	2.86
Total Conditions		
Any Condition	371619	54.08
>= 2 conditions	217327	31.62
>= 3 conditions	92827	13.51
>= 4 conditions	26295	3.83
>= 5 conditions	5382	0.78
>= 6 conditions	786	0.11
>= 7 conditions	73	0.01
>= 8 conditions	12	0.00
>= 9 conditions	0	0.00
>= 10 conditions	0	0.00
>= 11 conditions	0	0.00

Table 8: Details on the machine to generate this document.

Setting	Value
version	R version 4.5.2 (2025-10-31)
os	macOS Sonoma 14.8.3
system	x86_64, darwin20
CPU	Intel(R) Core(TM) i9-9880H CPU @ 2.30GHz
RAM	16.0 GB
date	2026-03-17
pandoc	3.9 @ /usr/local/bin/ (via rmarkdown)
quarto	1.9.35 @ /usr/local/bin/quarto

Table 9: R packages and versions used to generate this document.

Package	Version	Package	Version	Package	Version	Package	Version
backports	1.5.0	generics	0.1.4	otel	0.2.0	scales	1.4.0
bit	4.6.0	ggplot2	4.0.2	pccc	1.0.7	sessioninfo	1.2.3
bit64	4.6.0-1	glue	1.8.0	pillar	1.11.1	stringi	1.8.7
blob	1.3.0	gtable	0.3.6	pkgconfig	2.0.3	stringr	1.6.0
cachem	1.1.0	hms	1.1.4	qwraps2	0.6.2	svglite	2.2.2
checkmate	2.3.4	htmltools	0.5.9	R.methodsS3	1.8.2	systemfonts	1.3.2
cli	3.6.5	jsonlite	2.0.0	R.oo	1.27.1	textshaping	1.0.5
codetools	0.2-20	kableExtra	1.4.0	R.utils	2.13.0	tibble	3.3.1
comorbidity	1.1.0	knitr	1.51	R6	2.6.1	tidyselect	1.2.1
data.table	1.18.2.1	labeling	0.4.3	RColorBrewer	1.1-3	tinytex	0.58
DBI	1.3.0	lifecycle	1.0.5	Rcpp	1.1.1	vctrs	0.7.1
digest	0.6.39	magrittr	2.0.4	rlang	1.1.7	viridisLite	0.4.3
dplyr	1.2.0	medicalcoder	0.8.0	rmarkdown	2.30	withr	3.0.2
evaluate	1.0.5	memoise	2.0.1	RSQLite	2.4.6	xfun	0.56
farver	2.1.2	microbenchmark	1.5.0	rstudioapi	0.18.0	xml2	1.5.2
fastmap	1.2.0	odbc	1.6.4.1	S7	0.2.1	yaml	2.3.12

Feinstein, James A., Seth Russell, Peter E. DeWitt, Chris Feudtner, Dingwei Dai, and Tellen D. Bennett. 2018. "R Package for Pediatric Complex Chronic Condition Classification." *JAMA Pediatrics* 172 (6): 596–98. https://doi.org/10.1001/jamapediatrics.2018.0256.

Feudtner, Chris, James A Feinstein, Wenjun Zhong, Matt Hall, and Dingwei Dai. 2014. "Pediatric Complex Chronic Conditions Classification System Version 2: Updated for ICD-10 and Complex Medical Technology Dependence and Transplantation." *BMC Pediatrics* 14: 1–7. https://doi.org/10.1186/1471-2431-14-199.

Gasparini, Alessandro. 2018. "Comorbidity: An r Package for Computing Comorbidity Scores." *Journal of Open Source Software* 3: 648. https://doi.org/10.21105/joss.00648.

Glasheen, William P, Tristan Cordier, Rajiv Gumpina, Gil Haugh, Jared Davis, and Andrew Renda. 2019. "Charlson Comorbidity Index: ICD-9 Update and ICD-10 Translation." *American Health & Drug Benefits* 12 (4): 188. https://pubmed.ncbi.nlm.nih.gov/31428236/.

Goldberger, Ary L., Luis A. N. Amaral, Leon Glass, et al. 2000. "PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals." *Circulation* 101 (23): e215–20. https://doi.org/10.1161/01.cir.101.23.e215.

Healthcare Research, Agency for, and Quality (AHRQ). 2025. *Elixhauser Comorbidity Software Refined for ICD-10-CM Healthcare Cost and Utilization Project (HCUP)*. https://hcup-us.ahrq.gov/toolssoftware/comorbidityicd10/comorbidity_icd10.jsp.

Johnson, Alistair EW, Lucas Bulgarelli, Lu Shen, et al. 2023. "MIMIC-IV, a Freely Accessible Electronic Health Record Dataset." *Scientific Data* 10 (1): 1.

Johnson, Alistair, Lucas Bulgarelli, Tom Pollard, et al. 2024. "MIMIC-IV." *PhysioNet*, ahead of print, October. https://doi.org/10.13026/kpb9-mt58.

Quan, Hude, Bo Li, Colette M. Couris, et al. 2011. "Updating and Validating the Charlson Comorbidity Index and Score for Risk Adjustment in Hospital Discharge Abstracts Using Data from 6 Countries." *American Journal of Epidemiology* 173 (6): 676–82. https://doi.org/10.1093/aje/kwq433.

Quan, Hude, Vijaya Sundararajan, Patricia Halfon, et al. 2005. "Coding Algorithms for Defining Comorbidities in ICD-9-CM and ICD-10 Administrative Data." *Medical Care* 43 (11): 1130–39. https://doi.org/10.1097/01.mlr.0000182534.19832.83.

Xie, Yihui. 2014. "Knitr: A Comprehensive Tool for Reproducible Research in R." In *Implementing Reproducible Computational Research*, edited by Victoria Stodden, Friedrich Leisch, and Roger D. Peng. Chapman; Hall/CRC.

Xie, Yihui. 2015. *Dynamic Documents with R and Knitr*. 2nd ed. Chapman; Hall/CRC. https://yihui.org/knitr/.

Xie, Yihui. 2025. *Knitr: A General-Purpose Package for Dynamic Report Generation in R*. https://yihui.org/knitr/.

Zhu, Hao. 2024. *kableExtra: Construct Complex Table with 'Kable' and Pipe Syntax*. https://doi.org/10.32614/CRAN.package.kableExtra.